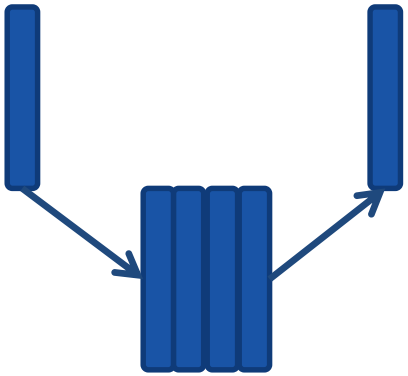
Föreläsning 4

Kö

Föreläsning 4

- ADT Kö
- Kö JCF
- Kö implementerad med en cirkulär array
- Kö implementerad med en länkad lista

ADT Kö



Grundprinciper: En kö fungerar som en kö.

Man fyller på den längst bak och tömmer den längst fram, FIFO - **F**irst **I**n **F**irst **O**ut.

Välanvänd datastruktur (printköer, processköer, problemlösning,…).

Måste-metoder:
enqueue(element)-köar element sist

dequeue()-tar bort första elementet i kön och returnerar detta

Vanliga metoder:

empty() – returnerar true om kön är tom

front() – returnerar första elementet utan att ta bort det

size() – returnerar antal element i kön

Och så behöver vi en create()…

Kö i JCF

I JCF är kö ett interface Queue<E>

- Ej konventionella namn på metoder
- För varje operation finns två metod:
  - en som returnerar null eller false om den misslyckas
  - en metod som kastar ett exception om den misslyckas

Att använda en kö i JCF

Queue implementeras av LinkedList och ArrayDeque (circulär array):

```
Queue<String> nameQ = new LinkedList<>;
Queue<String> nameQ = new ArrayDeque<>();
```

Nu kommer vi bara åt metoderna i interfacet på vår nameQ.
Dock kan vi fortfarande kasta om vår nameQ för att komma åt element mitt i kön:

```
String s= ((ArrayDeque<String>) nameQ).get(2);
```

Vill man undvika detta möjliga missbruk måste vi skapa en ny klass.
Det är ändå mycket bättre inkapslat än för Stack.

Oftast är ArrayDeque rätt val

Metoder i Queue

`add(E e)`

Inserts the specified element into this queue returning true upon success and throwing an IllegalStateException if no space is currently available.

`offer(E e)`

Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions. Returns true or false.

`remove()`

Retrieves and removes the head of this queue. Throws NoSuchElementException if this queue is empty.

`poll()`

Retrieves and removes the head of this queue, or returns null if this queue is empty.

`element()`

Retrieves, but does not remove, the head of this queue. Throws NoSuchElementException if this queue is empty.

`peek()`

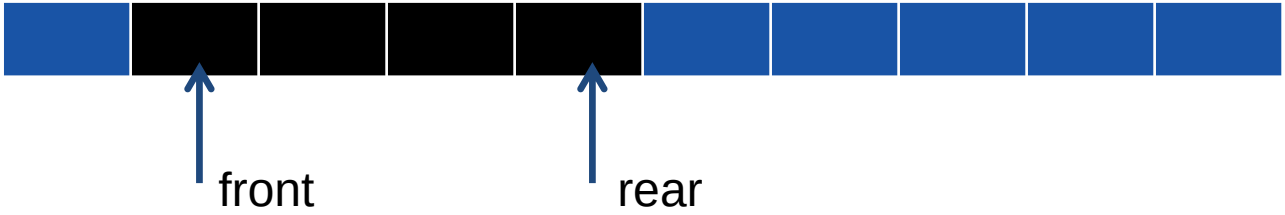
Retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.

Interfacet ärver bla `isEmpty` och `size`. Dessa måste också implementeras.

Kö implementerad med en cirkulär array

Att flytta alla element när vi tar bort första elementet (alternativt när vi köar ett element) blir ineffektivt - O(n)

Lösning: Cirkulär array, blir O(1) på alla operationer:



Dequeue: front++ Enqueue: rear++

Om rear = maxSize sätt rear = 0

Om front = maxSize sätt front = 0

rear får ej komma ifatt front → full array → reallocate

Vad ska rear och front ha för startvärden?

Steg 1

```
public class ArrayQueue<E> {
  private int front, rear, size, maxSize;
  private E[] data;

  public ArrayQueue(int initialMaxSize){
    size = 0;
    front = 0;
    maxSize = initialMaxSize;
    rear = maxSize-1;
    data = (E[]) new Object[maxSize];
  }

  public boolean offer(E element){
    // skriv kod som sätter in elementet, strunta i problemet att arrayen kan bli full
    return true;
  }
}
//Kompilera och testa att det verkar fungera att sätta in element
```

Lösningsförslag offer

```
public boolean offer(E element){
   rear = (rear+1) % maxSize;
   data[rear] = element;
   size++;
   return true;
}
```

Steg 2

Skriv en peek – metod som returnerar null om kön är tom

Testa att både offer och peek verkar fungera

Lösningsförslag peek

```
public E peek(){
   if(size==0) return null;
   return data[front];
}
```

Steg 3

Skriv sedan en poll metod.

Lösningsförslag poll

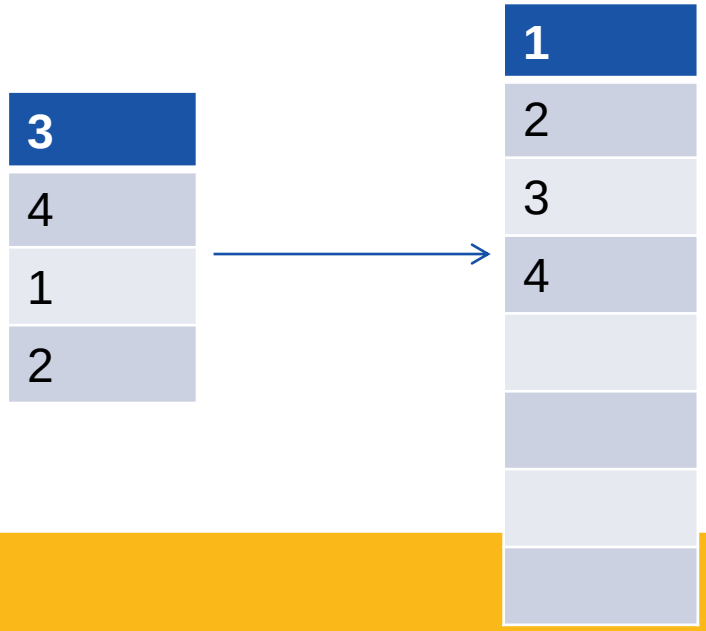
```
public E poll(){
   if(size==0) return null;
   size--;
   E element = data[front];
   front =(front+1) % maxSize;
   return element;
}
```

Steg 4

Dags att ta tag i problemet att arrayen kan bli full.

Som vanligt dubblar vi den så att alla operationer blir O(1) i snitt.

Vi måste ta hand om att kön kan ligga över arrayslutet:



Lösningsförslag reallocate

```
private void reallocate() {
    int newMaxSize = 2 * maxSize;
    E[] newData = (E[]) new Object[newMaxSize];
    int j = front;
    for (int i = 0; i < size; i++) {
        newData[i] = data[j];
        j = (j + 1) % maxSize;
    }
    front = 0;
    rear = size - 1;
    maxSize = newMaxSize;
    data = newData;
}
```

```
public boolean offer(E element) {
    if (size == maxSize) {
        reallocate();
    }
    rear = (rear + 1) % maxSize;
    data[rear] = element;
    size++;
    return true;
}
```

Testkod

Vilka olika mekanismer och fall behöver vi testa? Koden nedan testar absolut inte allt. Vad testar den? Vad mer behöver vi testa?

```
public class Test {
	public static void main(String[] args) {
		ArrayQueue<String> nameQ = new ArrayQueue<String>(10);
		for(int i=0;i<8;i++)
			nameQ.offer("e"+(i+1));
		nameQ.poll();
		nameQ.poll();
		for(int i=8;i<14;i++)
			nameQ.offer("e"+(i+1));
		while(nameQ.peek()!=null)
			System.out.println(nameQ.poll());
	}

}
```

Implementera interface Queue

- Snyggast vore att implementera interfacet Queue i vår kö.
- Vi måste då implementera alla metoder.
- Viss hjälp kan vi få genom att ärva från AbstractQueue som implementerar interfacet Queue
- Dock måste vi implementera en iterator
  - viktigt att vi inte implementerar remove vilket skulle bryta mot kö-principen
  - egentligen är även next tveksam som ger oss möjlighet att både få data från och påverka objekt längre ner i kön
- Vill vi ha en korrekt kö kan vi alltså inte implementera Queue interfacet i javas api

```java
import java.util.AbstractQueue;
import java.util.Iterator;
import java.util.NoSuchElementException;

public class ArrayQueue<E> extends AbstractQueue<E>{ // implements Queue<E> görs av AbstractQueue<E>
    private int front, rear, size, maxSize;
    private E[] data;

    @SuppressWarnings("unchecked")
    public ArrayQueue(int initialMaxSize) {
        size = 0;
        front = 0;
        maxSize = initialMaxSize;
        rear = maxSize - 1;
        data = (E[]) new Object[maxSize];
    }
    @Override
    public boolean offer(E element) {
        if (size == maxSize) {
            reallocate();
        }
        rear = (rear + 1) % maxSize;
        data[rear] = element;
        size++;
        return true;
    }
    public E peek() {
        if (size == 0) {
            return null;
        }
        return data[front];
    }
    public E poll() {
        if (size == 0) {
            return null;
        } else {
            size--;
            E element = data[front];
            front = (front + 1) % maxSize;
            return element;
        }
    }
```

```
@SuppressWarnings("unchecked")
  private void reallocate() {
      int newMaxSize = 2 * maxSize;
      E[] newData = (E[]) new Object[newMaxSize];
      int j = front;
      for (int i = 0; i < size; i++) {
          newData[i] = data[j];
          j = (j + 1) % maxSize;
      }
      front = 0;
      rear = size - 1;
      maxSize = newMaxSize;
      data = newData;
  }
  @Override
  public Iterator<E> iterator() {
      return new Iter();
  }
  @Override
  public int size() {
      return size;
  }
  private class Iter implements Iterator<E> {
      private int index;
      private int count = 0;
      public Iter() {
          index = front;
      }
      @Override
      public boolean hasNext() {
          return count < size;
      }
      @Override
      public E next() {
          if (!hasNext()) {
              throw new NoSuchElementException();
          }
          E returnValue = data[index];
          index = (index + 1) % maxSize;
          count++;
          return returnValue;
      }
      @Override
      public void remove() {
          //ska endast kunna ta bort det första
          throw new UnsupportedOperationException();
      }
  }
}
```

Kö implementerad med en länkad lista

- En länkad lista har fördelen att den alltid har rätt storlek på bekostnad av att den tar större plats pga länkarna.
- En enkellänkad kö tar lika stor plats som en array kö när denna är fylld till hälften. Om arrayen inte krymper när kön krymper kan den slösa mycket mer utrymme.
- Att sätta in i kön (sist i listan) är O(n) men det kan man lösa genom att ha en länk till sista elementet (tail).
- På en dubbellänkad lista är alla operationer O(1) men den tar ännu större plats (tre gånger så stor plats som en full array).
- Starkaste skälet för att välja en länkad lista är att köer ofta krymper.

Java har valt att låta LinkedList och ArrayDeque implementera *Queue* (ArrayDequeu är normalt snabbast pga att länkade listor är långsamma).